

## **INGENIERÍA EN ENTORNOS VIRTUALES Y NEGOCIOS DIGITALES**

*Materia: Programación para Entornos Virtuales*

*Profesor(a): Ing. Anastasio Rodríguez García*

*Curso NetAcad*

*Fundamentos de Python 2*

*Grupo: GIEV0351*

*Alumno:*

**Aguillon Valdez Luis Angel**

## Indice

# Modulo 1

Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

PE2: Módulo 1. Módulos, Paquetes, y PIP100%

1.0. Bienvenido a Fundamentos de Python 23/3

1.0.1 Acerca del Curso

1.0.2 Temario

1.0.3 Prepárate para el examen PCAP-31-0x

1.1. Sección 1 - Introducción a los módulos en Python10/10

1.2. Sección 2 - Módulos Selectos de Python (math, random, platform)9/9

1.3. Sección 3 - Módulos y Paquetes5/5

1.4. Sección 4 - Instalador de paquetes de Python (PIP)9/9

1.5. Finalización del Módulo 1 - Prueba del Módulo

PE2: Módulo 2. Cadenas, Métodos de Cadenas y listas, Excepciones98%

PE2: Módulo 3. Programación Orientada a Objetos100%

PE2: Módulo 4. Misceláneos85%

Prueba Final del curso Fundamentos de Python 2 (PE2)100%

1.0.1 Acerca del Curso



1.0 Bienvenido a Fundamentos de Python 2

Detallar para comenzar

Este curso es el segundo en una serie de dos cursos, y le prepara para el examen de certificación [PCAP\\_Certified Associate in Python](#) a través de Pearson VUE.

Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

PE2: Módulo 1. Módulos, Paquetes, y PIP100%

1.0. Bienvenido a Fundamentos de Python 23/3

1.1. Sección 1 - Introducción a los módulos en Python10/10

1.1.1 ¿Qué es un módulo?

1.1.2 ¿Cómo hacer uso de un módulo?

1.1.3 Importando un módulo

1.1.4 Namespace

1.1.5 Importando un módulo: continuación

1.1.6 Importando un módulo: \*

1.1.7 La palabra clave as

1.1.8 Aliasing (renombrado)

1.1.9 RESUMEN DE LA SECCIÓN

1.1.10 CUESTIONARIO DE SECCIÓN

1.2. Sección 2 - Módulos Selectos de Python (math, random, platform)9/9

1.3. Sección 3 - Módulos y Paquetes5/5

1.4. Sección 4 - Instalador de paquetes de Python (PIP)9/9

1.5. Finalización del Módulo 1 - Prueba del Módulo

PE2: Módulo 2. Cadenas, Métodos de Cadenas y listas, Excepciones98%

1.1.4 Namespace

explicación será lo más breve posible.

Un **namespace** es un espacio (entendido en un contexto no físico) en el que existen algunos nombres y los nombres no entran en conflicto entre sí (es decir, no hay dos objetos diferentes con el mismo nombre). Podemos decir que cada grupo social es un namespace - el grupo tiende a nombrar a cada uno de sus miembros de una manera única (por ejemplo, los padres no darían a sus hijos el mismo nombre).

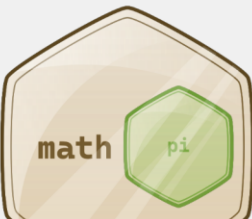
| JOHNSON                           | SMITH                                   | CURIE                                   |
|-----------------------------------|-----------------------------------------|-----------------------------------------|
| Carl<br>Beverly<br>Brian<br>Sweet | Adam<br>Brad<br>Frank<br>Janet<br>Agent | Marie<br>Irène<br>Ève<br>Pierre<br>Hara |

Esta singularidad se puede lograr de muchas maneras, por ejemplo, mediante el uso de apodos junto con los nombres (funcionará dentro de un grupo pequeño como una clase en una escuela) o asignando identificadores especiales a todos los miembros del grupo (el número de Seguro Social de EE. UU. es un buen ejemplo de tal práctica).

Dentro de un determinado namespace, cada nombre debe permanecer único. Esto puede significar que algunos nombres pueden desaparecer cuando cualquier otra entidad de un nombre ya conocido ingresa al namespace. Mostraremos como funciona y como controlarlo, pero primero, volvamos a las importaciones.

Si el módulo de un nombre especificado **existe y es accesible** (un módulo es de hecho un **archivo fuente de Python**), Python importa su contenido, **se hacen conocidos todos los nombres definidos en el módulo**, pero no ingresan al namespace del código.

Esto significa que puedes tener tus propias entidades llamadas





Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

Excepciones88%

2.1. Sección 1 - Caracteres y Cadenas versus las Computadoras3 / 5

2.1.1 Como las computadoras entienden los caracteres individuales

2.1.2 ISBN

2.1.3 Puntos de código y páginas de código

2.1.4 RESUMEN DE SECCIÓN

2.1.5 CUESTIONARIO DE SECCIÓN

2.2. Sección 2 - La naturaleza de las cadenas en Python10 / 10

2.3. Sección 3 - Métodos de cadenas28 / 28

2.4. Sección 4 - Cadenas en acción6 / 6

2.5. Sección 5 - Cuatro programas simples11 / 11

2.6. Sección 6 - Errores: el pan diario del programador4 / 4

2.7. Sección 7 - La anatomía de las excepciones3 / 3

2.8. Sección 8 - Excepciones útiles4 / 4

2.9. Finalización del Módulo 2 - Prueba del Módulo

PE2: Módulo 3. Programación Orientada a Objetos100%

PE2: Módulo 4. Misceláneos85%

2.1.5 CUESTIONARIO DE SECCIÓN

Pregunta 1: ¿Qué es BOM?

Ocultar

BOM (Byte Order Mark), una Marca de Orden de Bytes es una combinación especial de bits que anuncia la codificación utilizada por el contenido de un archivo (por ejemplo, UCS-4 o UTF-8).

Pregunta 2: ¿Está Python 3 internacionalizado?

Ocultar

Si, está completamente internacionalizado: podemos usar caracteres UNICODE dentro de nuestro código, leerlos desde la entrada y enviarlos a la salida.

Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

PE2: Módulo 1. Módulos, Paquetes, y PIP100%

PE2: Módulo 2. Cadenas, Métodos de Cadenas y listas, Excepciones88%

2.1. Sección 1 - Caracteres y Cadenas versus las Computadoras3 / 5

2.2. Sección 2 - La naturaleza de las cadenas en Python10 / 10

2.2.1 Cadenas: un breve repaso

2.2.2 Cadenas multilinea

2.2.3 Operaciones con cadenas

2.2.4 Cadenas como secuencias

2.2.5 Referencias

2.2.6 Las operadores in y not in

2.2.7 Las cadenas en Python son inmutables

2.2.8 Operaciones con cadenas: continuación

2.2.9 RESUMEN DE SECCIÓN

2.2.10 CUESTIONARIO DE SECCIÓN

2.3. Sección 3 - Métodos de cadenas28 / 28

2.4. Sección 4 - Cadenas en acción6 / 6

2.5. Sección 5 - Cuatro programas simples11 / 11

2.6. Sección 6 - Errores: el pan diario del programador4 / 4

2.7. Sección 7 - La anatomía de las excepciones3 / 3

2.2.8 Operaciones con cadenas: continuación

min()

Ahora que comprendes que las cadenas son secuencias, podemos mostrarte algunas capacidades de secuencia menos obvias. Las presentaremos utilizando cadenas, pero no olvides que las listas también pueden adoptar los mismos trucos.

Comenzaremos con la función llamada `min()`.

Esta función encuentra el elemento mínimo de la secuencia pasada como argumento. Existe una condición - la secuencia (cadena o lista) no puede estar vacía, de lo contrario obtendrás una excepción `ValueError`.

```
1 # Demostración de min() - Ejemplo 1:
2 print(min("aBbByZz"))
3
4
5 # Demostración de min() - Ejemplos 2 y 3:
6 t = "Los Caballeros Que Dicen "Mi""
7 print('[' + min(t) + ']')
8
9 t = [0, 1, 2]
10 print(min(t))
11
```

El programa Ejemplo 1 da la siguiente salida:

Output

Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

PE2: Módulo 2. Cadenas, Métodos de Cadenas y listas, Excepciones88%

2.1. Sección 1 - Caracteres y Cadenas versus las Computadoras3 / 5

2.2. Sección 2 - La naturaleza de las cadenas en Python10 / 10

2.3. Sección 3 - Métodos de cadenas28 / 28

2.4. Sección 4 - Cadenas en acción6 / 6

2.4.1 Comparando cadenas

2.4.2 Ordenamiento

2.4.3 Cadenas versus números

2.4.4 RESUMEN DE SECCIÓN

2.4.5 CUESTIONARIO DE SECCIÓN

2.4.6 LAB Un Display LED

2.5. Sección 5 - Cuatro programas simples11 / 11

2.6. Sección 6 - Errores: el pan diario del programador4 / 4

2.7. Sección 7 - La anatomía de las excepciones3 / 3

2.8. Sección 8 - Excepciones útiles4 / 4

2.9. Finalización del Módulo 2 - Prueba del Módulo

PE2: Módulo 3. Programación Orientada a Objetos

2.4.3 Cadenas versus números

2.4.3 Cadenas versus números

Hay dos cuestiones adicionales que deberían discutirse aquí: ¿Cómo convertir un número (un entero o un flotante) en una cadena, y viceversa? Puede ser necesario realizar tal transformación. Además, es una forma rutinaria de procesar datos de entrada o salida.

La conversión de cadena a número es simple, ya que siempre es posible. Se realiza mediante una función llamada `int()`.

Justo como aquí:

```
1 itg = 13
2 f1t = 1.3
3 si = str(itg)
4 sf = str(f1t)
5
6 print(si + ' ' + sf)
7
```

La salida del código es:

Output

Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

PE2: Módulo 2. Cadenas, Métodos de Cadenas y listas, Excepciones100%

2.1. Sección 1 - Caracteres y Cadenas versus las Computadoras5 / 5

2.2. Sección 2 - La naturaleza de las cadenas en Python10 / 10

2.3. Sección 3 - Métodos de cadenas28 / 28

2.4. Sección 4 - Cadenas en acción8 / 8

2.5. Sección 5 - Cuatro programas simples11 / 11

2.6. Sección 6 - Errores: el pan diario del programador4 / 4

2.6.1 Errores, fallas y otras plagas

2.6.2 Excepciones

2.6.3 RESUMEN DE SECCIÓN

2.6.4 CUESTIONARIO DE SECCIÓN

2.7. Sección 7 - La anatomía de las excepciones3 / 3

2.8. Sección 8 - Excepciones útiles4 / 4

2.9. Finalización del Módulo 2 - Prueba del Módulo

PE2: Módulo 3. Programación Orientada a Objetos100%

PE2: Módulo 4. Misceláneos85%

2.6.1 Errores, fallas y otras plagas

2.6 Sección 6 - Errores: el pan diario del programador

2.6.1 Errores, fallas y otras plagas

Cualquier cosa que pueda salir mal, saldrá mal.

Esta es la ley de Murphy, y funciona en todo y siempre. Si la ejecución del código puede salir mal, lo hará.

Observa el código en el editor. Hay al menos dos formas posibles de que "salga mal" la ejecución. ¿Puedes verlas?

Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

2.1. Sección 1 - Caracteres y Cadenas versus las Computadoras5 / 5

2.2. Sección 2 - La naturaleza de las cadenas en Python10 / 10

2.3. Sección 3 - Métodos de cadenas28 / 28

2.4. Sección 4 - Cadenas en acción8 / 8

2.5. Sección 5 - Cuatro programas simples11 / 11

2.6. Sección 6 - Errores: el pan diario del programador4 / 4

2.7. Sección 7 - La anatomía de las excepciones3 / 3

2.8. Sección 8 - Excepciones útiles4 / 4

2.8.1 Excepciones integradas

2.8.2 LAB Leyendo enteros de manera segura

2.8.3 RESUMEN DE SECCIÓN

2.8.4 CUESTIONARIO DE SECCIÓN

2.9. Finalización del Módulo 2 - Prueba del Módulo

PE2: Módulo 3. Programación Orientada a Objetos100%

PE2: Módulo 4. Misceláneos85%

Prueba final del curso Fundamentos de Python 2 (PE2)100%

2.8.2 LAB Leyendo enteros de manera segura

Tu tarea es escribir una función capaz de ingresar valores enteros y verificar si están dentro de un rango especificado.

La función deberá:

- Aceptar tres argumentos: una entrada, un límite inferior aceptable y un límite superior aceptable.
- Si el usuario ingresa una cadena que no es un valor entero, la función debe emitir el mensaje `Error: entrada incorrecta` y solicitará al usuario que ingrese el valor nuevamente.
- Si el usuario ingresa un número que está fuera del rango especificado, la función debe emitir el mensaje `Error: el valor no está dentro del rango permitido (min..max)` y solicitará al usuario que ingrese el valor nuevamente.
- Si el valor de entrada es válido, será regresado como resultado.

Datos de Prueba

Prueba tu código cuidadosamente.

Así es como la función debería reaccionar ante la entrada del usuario:

```
Ingresar un número entre -10 a 10: 100
Error: el valor no está dentro del rango permitido (-10..10)
Ingresar un número entre -10 a 10: asd
Error: entrada incorrecta
Ingresar un número entre -10 a 10: 1
El número es: 1
```

```
1 def read_int(prompt, min, max):
2     #
3     # Escribe tu código aquí.
4     #
5
```

## Modulo 3

Curso  
Networking  
Academy

Python

Fundamentos de Python 2

Course Outline

Resources

Search course outline

Excepciones

PE2: Módulo 3. Programación Orientada a Objetos

3.1. Sección 1 - Los fundamentos de la POO

3.1.1 Los conceptos básicos del enfoque orientado a objetos

3.1.2 Enfoque procedimental versus el enfoque orientado a objetos

3.1.3 Jerarquía de clases

3.1.4 ¿Qué es un objeto?

3.1.5 Herencia

3.1.6 ¿Qué contiene un objeto?

3.1.7 Tu primera clase

3.1.8 Tu primer objeto

3.1.9 RESUMEN DE SECCIÓN

3.1.10 CUESTIONARIO DE SECCIÓN

3.2. Sección 2 - Un corto viaje desde el enfoque procedimental hacia el orientado a objetos

3.3. Sección 3 - POO: Propiedades

3.4. Sección 4 - POO: Métodos

3.5. Sección 5 - Fundamentos de la POO: Herencia

3.6. Sección 6 - Más sobre excepciones

3.7. Finalización del Módulo 3 - Prueba del Módulo

3.1.3 Jerarquía de clases

La palabra **clases** tiene muchos significados, pero no todos son compatibles con las ideas que queremos discutir aquí. La clase que nos concierne es como una **categoría**, como resultado de similitudes definidas con precisión. Intentaremos señalar algunas clases que son buenos ejemplos de este concepto.

```
graph BT; VT[Vehículos Terrestres]; VA[Vehículos Acuáticos]; VAero[Vehículos Aéreos]; VE[Vehículos Espaciales]; VCR[Vehículos con ruedas]; VCP[Vehículos con pistas]; A[Aerodispositivos]; V[Vehículos]; VCR --> VT; VCR --> VA; VCP --> VT; VCP --> VA; A --> VAero; A --> VE; VT --> V; VA --> V; VAero --> V; VE --> V;
```

Curso  
Networking  
Academy

Python

Fundamentos de Python 2

Course Outline

Resources

Search course outline

Excepciones

PE2: Módulo 3. Programación Orientada a Objetos

3.1. Sección 1 - Los fundamentos de la POO

3.1.1 Los conceptos básicos del enfoque orientado a objetos

3.1.2 Enfoque procedimental versus el enfoque orientado a objetos

3.1.3 Jerarquía de clases

3.1.4 ¿Qué es un objeto?

3.1.5 Herencia

3.1.6 ¿Qué contiene un objeto?

3.1.7 Tu primera clase

3.1.8 Tu primer objeto

3.1.9 RESUMEN DE SECCIÓN

3.1.10 CUESTIONARIO DE SECCIÓN

3.2. Sección 2 - Un corto viaje desde el enfoque procedimental hacia el orientado a objetos

3.3. Sección 3 - POO: Propiedades

3.4. Sección 4 - POO: Métodos

3.5. Sección 5 - Fundamentos de la POO: Herencia

3.6. Sección 6 - Más sobre excepciones

3.7. Finalización del Módulo 3 - Prueba del Módulo

3.1.7 Tu primera clase

La programación orientada a objetos es el **arte de definir y expandir clases**. Una clase es un modelo de una parte muy específica de la realidad, que refleja las propiedades y actividades que se encuentran en el mundo real.

Las clases definidas al principio son demasiado generales e imprecisas para cubrir el mayor número posible de casos reales.

No hay obstáculo para definir nuevas subclases más precisas. Heredarán todo de su superclase, por lo que el trabajo que se utilizó para su creación no se desperdicia.

La nueva clase puede agregar nuevas propiedades y nuevas actividades y, por lo tanto, puede ser más útil en aplicaciones específicas. Obviamente, se puede usar como una superclase para cualquier número de subclases recién creadas.

El proceso no necesita tener un final. Puedes crear tantas clases como necesites.

La clase que se define no tiene nada que ver con el objeto: **la existencia de una clase no significa que ninguno de los objetos compatibles se creará automáticamente**. La clase en sí misma no puede crear un objeto: debes crearlo tu mismo y Python te permite hacerlo.

Es hora de definir la clase más simple y crear un objeto. Analiza el siguiente ejemplo:

```
1 class TheSimplestClass:
2     pass
3
```

Hemos definido una clase. La clase es bastante pobre: no contiene propiedades ni actividades. Esta **vacía**, pero eso no importa por ahora. Cuanto más simple sea la clase, mejor para nuestros propósitos.

La **definición comienza con la palabra clave reservada `class`**. La palabra clave reservada es seguida por un **identificador que le dará nombre a la clase** (nota: no lo confundas con el nombre del objeto: estas son dos cosas diferentes).

A continuación, se agregan **dos puntos**, como clases, como funciones, forman su propio bloque anidado. El contenido dentro del bloque define todas las propiedades y actividades de la clase.

La palabra clave reservada `pass` llena la clase con nada. No contiene ningún método ni propiedades.

Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

PE2: Módulo 2. Cadenas, Métodos de Cadenas y listas, Excepciones88%

PE2: Módulo 3. Programación Orientada a Objetos100%

3.1. Sección 1 - Los fundamentos de la POO10 / 10

3.2. Sección 2 - Un corto viaje desde el enfoque procedimental hacia el orientado a objetos10 / 10

3.2.1 ¿Qué es una pila?

3.2.2 La pila: el enfoque procedimental

3.2.3 La pila: el enfoque procedimental versus el enfoque orientado a objetos

3.2.4 La pila: el enfoque orientado a objetos

3.2.5 El enfoque orientado a objetos: una pila desde cero

3.2.6 RESUMEN DE SECCIÓN

3.2.7 CUESTIONARIO DE SECCIÓN

3.2.8 LAB Pila contadora

3.2.9 LAB Cotas alias FIFO

3.2.10 LAB Cotas alias FIFO: parte 2

3.3. Sección 3 - POO: Propiedades5 / 5

3.4. Sección 4 - POO: Métodos10 / 10

3.5. Sección 5 - Fundamentos de la POO: Herencia11 / 11

3.6. Sección 6 - Más sobre excepciones6 / 6

3.7. Finalización del Módulo 3 - Prueba del Módulo

3.2.5 El enfoque orientado a objetos: una pila desde cero

Ahora es el momento de que las dos funciones (métodos) implementen las operaciones push y pop. Python supone que una función de este tipo debería estar **inmersa dentro del cuerpo de la clase**, como el constructor.

Queremos invocar estas funciones para `self.stack` y `self.val` valores de la pila. Esto significa que ambos deben ser accesibles para el usuario de la clase (en contraste con la lista previamente construida, que está oculta para los usuarios de la clase ordinaria).

Tal componente es llamado **público**, por ello **no puede comenzar su nombre con dos (o más) guilanes bajos**. Hay un requisito más: **el nombre no debe tener más de un guión bajo**. Dado que ningún guión bajo final cumple completamente con el requisito, puedes suponer que el nombre es aceptable.

Las funciones en sí son simples. Echa un vistazo

```
1 class Stack:
2     def __init__(self):
3         self.__stack_list = []
4
5     def push(self, val):
6         self.__stack_list.append(val)
7
8     def pop(self):
9         val = self.__stack_list[-1]
10        del self.__stack_list[-1]
11        return val
```

Sin embargo, hay algo realmente extraño en el código. Las funciones parecen familiares, pero tienen más parámetros que sus contrapartes procedimentales.

Aquí, ambas funciones tienen un parámetro llamado `self` en la primera posición de la lista de parámetros.

Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

PE2: Módulo 3. Programación Orientada a Objetos100%

3.1. Sección 1 - Los fundamentos de la POO10 / 10

3.2. Sección 2 - Un corto viaje desde el enfoque procedimental hacia el orientado a objetos10 / 10

3.3. Sección 3 - POO: Propiedades5 / 5

3.4. Sección 4 - POO: Métodos10 / 10

3.4.1 Métodos a detalle

3.4.2 La vida al interior de las clases y objetos

3.4.3 Reflexión e introspección

3.4.4 Investigando Clases

3.4.5 RESUMEN DE SECCIÓN

3.4.6 CUESTIONARIO DE SECCIÓN

3.4.7 LAB La clase Timer

3.4.8 LAB Das de la semana

3.4.9 LAB Puntos en un plano

3.4.10 LAB Triángulo

3.5. Sección 5 - Fundamentos de la POO: Herencia11 / 11

3.6. Sección 6 - Más sobre excepciones6 / 6

3.7. Finalización del Módulo 3 - Prueba del Módulo

3.4.2 La vida al interior de las clases y objetos

Cada clase de Python y cada objeto de Python está pre-equipado con un conjunto de atributos útiles que pueden usarse para examinar sus capacidades.

Ya conoces uno de estos: es la propiedad `__dict__`.

Observemos como esta propiedad trata con los métodos: mira el código en el editor.

```
1 class Classy:
2     varia = 1
3     def __init__(self):
4         self.var = 2
5
6     def method(self):
7         pass
8
9     def __hidden(self):
10        pass
11
12 obj = Classy()
```

Ejecítalo para ver que produce. Verifica el resultado.

Encuentra todos los métodos y atributos definidos. Localiza el contexto en el que existen: dentro del objeto o dentro de la clase.

`__dict__` es un diccionario. Otra propiedad incorporada que vale la pena mencionar es una cadena llamada `__name__`.

La propiedad contiene **el nombre de la clase**. No es nada emocionante, es solo una cadena.



Cisco  
Networking  
Academy

Python

Fundamentos de Python 2

Course Outline

Resources

Search course outline

100%

PE2: Módulo 4. Misceláneos

85%

4.1. Sección 1 - generadores, iteradores y cierres

11 / 11

4.1.1 Generadores: donde encontrarlos

4.1.2 La sentencia yield

4.1.3 Cómo construir un generador

4.1.4 Más acerca de listas por comprensión

4.1.5 La función lambda

4.1.6 ¿Cómo usar lambdas y para qué?

4.1.7 Lambdas y la función map()

4.1.8 Lambdas y la función filter()

4.1.9 Una breve explicación de cierres

4.1.10 RESUMEN DE SECCIÓN

4.1.11 CUESTIONARIO DE SECCIÓN

4.2. Sección 2 - Archivos (Flujos de archivos, procesamiento de archivos, diagnóstico de problemas de flujo)

12 / 12

4.3. Sección 3 - Trabajando con archivos reales

12 / 12

4.4. Sección 4 - El módulo os - interactuando con el sistema operativo

10 / 10

4.5. Sección 5 - el módulo datetime - trabajar con funciones relacionadas con la hora y la fecha

22 / 22

4.6. Sección 6 - El módulo calendar - trabajando con funciones relacionadas con el calendario

15 / 15

4.7. Finalización del Módulo 4 - Prueba del Módulo

4.1.4 Más acerca de listas por comprensión

4.1.4 Más acerca de listas por comprensión

Debes poder recordar las reglas que rigen la creación y el uso de un fenómeno de Python llamado **listas por comprensión**: una forma simple de **crear listas y sus contenidos**.

En caso de que lo necesites, te proporcionamos un recordatorio en el editor.

```
1 list_1 = []
2
3 for ex in range(10):
4     list_1.append(10 ** ex)
5
6 list_2 = [10 ** ex for ex in range(10)]
7
8 print(list_1)
9 print(list_2)
10
```

Console

Existen dos partes dentro del código, ambas crean una lista que contiene algunas de las primeras potencias naturales de diez.

La primera parte utiliza una forma rutilaria del bucle `for`, mientras que la segunda hace uso de listas por comprensión y construye la lista en el momento, sin necesidad de un bucle o cualquier otro código.

Pareciera que la lista se crea dentro de sí misma, esto es falso, ya que Python tiene que realizar casi las mismas operaciones que en la primera parte, pero el segundo formalismo es simplemente más elegante y le evita al lector cualquier detalle innecesario.

El ejemplo genera dos líneas idénticas que contienen el siguiente texto:

Output

Cisco  
Networking  
Academy

Python

Fundamentos de Python 2

Course Outline

Resources

Search course outline

100%

PE2: Módulo 3. Programación Orientada a Objetos

100%

PE2: Módulo 4. Misceláneos

85%

4.1. Sección 1 - generadores, iteradores y cierres

11 / 11

4.2. Sección 2 - Archivos (Flujos de archivos, procesamiento de archivos, diagnóstico de problemas de flujo)

12 / 12

4.2.1 Accediendo archivos desde código en Python

4.2.2 Nombre de archivos

4.2.3 Flujos de archivos (streams)

4.2.4 Manejo de archivos

4.2.5 Abriendo los flujos o streams

4.2.6 Selección de los modos de texto y binario

4.2.7 Abriendo el flujo (stream) por primera vez

4.2.8 Flujos o streams pre-abiertos

4.2.9 Cerrando los flujos (streams)

4.2.10 Diagnosticando problemas con los flujos (streams)

4.2.11 RESUMEN DE SECCIÓN

4.2.12 CUESTIONARIO DE SECCIÓN

4.3. Sección 3 - Trabajando con archivos reales

12 / 12

4.4. Sección 4 - El módulo os - interactuando con el sistema operativo

10 / 10

4.5. Sección 5 - el módulo datetime - trabajar con funciones relacionadas con la hora y la fecha

22 / 22

4.6. Sección 6 - El módulo calendar - trabajando con funciones relacionadas con el calendario

15 / 15

4.2.3 Flujos de archivos (streams)

4.2.4 Manejo de archivos

Python supone que **cada archivo está oculto detrás de un objeto de una clase adecuada**.

Por supuesto, es difícil no preguntar cómo interpretar la palabra adecuada.

Los archivos se pueden procesar de muchas maneras diferentes: algunos dependen del contenido del archivo, otros de las intenciones del programador.

En cualquier caso, diferentes archivos pueden requerir diferentes conjuntos de operaciones y comportarse de diferentes maneras.

Un objeto de una clase adecuada es **creado cuando abres el archivo y lo aniquilas al momento de cerrarlo**.

Entre estos dos eventos, puedes usar el objeto para especificar que operaciones se deben realizar en un stream en particular. Las operaciones que puedes usar están impuestas por la **forma en que abriste el archivo**.

En general, el objeto proviene de una de las clases que se muestran aquí:

IOBase

RawIOBase

BufferedIOBase

TextIOBase

Nota: nunca se utiliza el constructor para dar vida a estos objetos. La única forma de **obtenerlos es invocar la función llamada `open()`**

Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

PE2: Módulo 4. Misceláneos85%

4.1. Sección 1 - generadores, iteradores y cierres11 / 11

4.2. Sección 2 - Archivos (Flujos de archivos, procesamiento de archivos, diagnóstico de problemas de flujo)12 / 12

4.3. Sección 3 - Trabajando con archivos reales12 / 12

- 4.3.1 Procesamiento de archivos de texto
- 4.3.2 readlines()
- 4.3.3 readlines()
- 4.3.4 Manejo de archivos de texto: write()
- 4.3.5 ¿Qué es un bytearray?
- 4.3.6 Cómo leer bytes de un flujo (stream)
- 4.3.7 Copiando archivos: una herramienta simple y funcional
- 4.3.8 LAB Histograma de frecuencia de caracteres
- 4.3.9 LAB Histograma de frecuencia de caracteres ordenado
- 4.3.10 LAB Evaluando los resultados de los estudiantes
- 4.3.11 RESUMEN DE SECCIÓN
- 4.3.12 CUESTIONARIO DE SECCIÓN

4.4. Sección 4 - El módulo os - interactuando con el sistema operativo10 / 10

4.5. Sección 5 - el módulo datetime - trabajar con funciones relacionadas con la hora y la fecha22 / 22

4.6. Sección 6 - El módulo calendar - trabajando con funciones relacionadas con el calendario10 / 10

4.7. Finalización del Módulo 4 - Prueba del Módulo

4.3.5 ¿Qué es un bytearray?

Antes de comenzar a hablar sobre archivos binarios, tenemos que informarte sobre una de las **clases especializadas que Python usa para almacenar datos amorfos**.

Los **datos amorfos son datos que no tienen forma específica**, son solo una serie de bytes.

Esto no significa que estos bytes no puedan tener su propio significado o que no puedan representar ningún objeto útil, por ejemplo, gráficos de mapa de bits.

Lo más importante de esto es que en el lugar donde tenemos contacto con los datos, no podemos, o simplemente no queremos, saber nada al respecto.

Los datos amorfos no pueden almacenarse utilizando ninguno de los medios presentados anteriormente: no son cadenas ni listas.

Debe haber un contenedor especial capaz de manejar dichos datos.

Python tiene más de un contenedor, uno de ellos es una **clase especializada llamada bytearray**, como su nombre indica, es un **arreglo que contiene bytes (amorfos)**.

Si deseas tener dicho contenedor, por ejemplo, para leer una imagen de mapa de bits y procesarla de alguna manera, debes crearlo explícitamente, utilizando uno de los constructores disponibles.

Observa:

```
1 data = bytearray(10)
2
```

Tal invocación crea un objeto bytearray capaz de almacenar diez bytes.

Nota: dicho constructor **llena todo el arreglo con ceros**.

Bytearrays se asemejan a listas en muchos aspectos. Por ejemplo, son **mutables**, son susceptibles a la función `len()` y puedes acceder a cualquiera de sus elementos usando indexación convencional.

Existe una limitación importante: **no debes establecer ningún elemento del arreglo de bytes con un valor que no sea un entero** (violar

Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

PE2: Módulo 4. Misceláneos85%

4.1. Sección 1 - generadores, iteradores y cierres11 / 11

4.2. Sección 2 - Archivos (Flujos de archivos, procesamiento de archivos, diagnóstico de problemas de flujo)12 / 12

4.3. Sección 3 - Trabajando con archivos reales12 / 12

- 4.3.1 Procesamiento de archivos de texto
- 4.3.2 readlines()
- 4.3.3 readlines()
- 4.3.4 Manejo de archivos de texto: write()
- 4.3.5 ¿Qué es un bytearray?
- 4.3.6 Cómo leer bytes de un flujo (stream)
- 4.3.7 Copiando archivos: una herramienta simple y funcional
- 4.3.8 LAB Histograma de frecuencia de caracteres
- 4.3.9 LAB Histograma de frecuencia de caracteres ordenado
- 4.3.10 LAB Evaluando los resultados de los estudiantes
- 4.3.11 RESUMEN DE SECCIÓN
- 4.3.12 CUESTIONARIO DE SECCIÓN

4.4. Sección 4 - El módulo os - interactuando con el sistema operativo10 / 10

4.5. Sección 5 - el módulo datetime - trabajar con funciones relacionadas con la hora y la fecha22 / 22

4.6. Sección 6 - El módulo calendar - trabajando con funciones relacionadas con el calendario10 / 10

4.7. Finalización del Módulo 4 - Prueba del Módulo

4.3.12 CUESTIONARIO DE SECCIÓN

Pregunta 1: ¿Qué se espera del método `readlines()` cuando el stream está asociado con un archivo vacío?

Ocultar

Una lista vacía (una lista de longitud cero).

Pregunta 2: ¿Qué se pretende hacer con el siguiente código?

```
for line in open("file", "rt"):
    for char in line:
        if char.lower() not in "aeiouy ":
            print(char, end="")
```

Ocultar

Copia el contenido del archivo `file` hacia la consola, ignorando las vocales.

Pregunta 3: Vas a procesar un mapa de bits almacenado en un archivo llamado `image.png` y quieres leer su contenido como un todo en una variable `bytearray` llamada `image`. Agrega una línea al siguiente código para lograr este objetivo.

```
try:
    stream = open("image.png", "rb")
    # Inserta una línea aquí.
    stream.close()
except IOError:
    print("fallido")
else:
```

Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

4.3. Sección 3 - Trabajando con archivos reales12 / 12

4.4. Sección 4 - El módulo os - interactuando con el sistema operativo10 / 10

4.5. Sección 5 - el módulo datetime - trabajar con funciones relacionadas con la hora y la fecha22 / 22

4.6. Sección 6 - El módulo calendar - trabajando con funciones relacionadas con el calendario10 / 10

- 4.6.1 Introducción al módulo calendar
- 4.6.2 Tu primer calendario
- 4.6.3 El calendario para un mes específico
- 4.6.4 La función `setfirstweekday()`
- 4.6.5 La función `weekday()`
- 4.6.6 La función `weekheader()`
- 4.6.7 ¿Cómo comprobamos si un año es bisiesto?
- 4.6.8 Clases para la creación de calendarios
- 4.6.9 Creando un objeto `Calendar`
- 4.6.10 El método `itermonthdays()`
- 4.6.11 Otros métodos que retornan o devuelven iteradores
- 4.6.12 El método `monthdays2calendar()`
- 4.6.13 LAB El módulo calendar
- 4.6.14 RESUMEN DE SECCIÓN
- 4.6.15 CUESTIONARIO DE SECCIÓN

4.7. Finalización del Módulo 4 - Prueba del Módulo

4.6.2 Tu primer calendario

Comenzarás tu aventura con el módulo `calendar` con una función simple llamada `calendar`, que te permite **mostrar el calendario para todo el año**. Veamos cómo usarlo para mostrar el calendario de 2020. Ejecuta el código en el editor y ve qué sucede.

```
1 import calendar
2 print(calendar.calendar(2020))
3
```

El resultado mostrado es similar al resultado del comando `cal` disponible en Unix. Si deseas cambiar el formato de calendario predeterminado, puedes utilizar los siguientes parámetros:

- `width`: ancho de la columna de fecha (por defecto 2).
- `l`: número de líneas por semana (por defecto 1).
- `mon`: número de espacios entre las columnas del mes (por defecto 6).
- `colwidth`: número de columnas (por defecto 3).

La función de calendario requiere que se especifique el año, mientras que los otros parámetros responsables del formato son opcionales. Te recomendamos que pruebes estos parámetros tu mismo.

Una buena alternativa a la función anterior es la función llamada `prcal`, que también toma los mismos parámetros que la función `calendar`, pero no requiere el uso de la función `print` para mostrar el calendario. Su uso se ve así: